

VIP 仿真验证系统 使用说明书

属性	值
版本	1.0.0
日期	2026-06-12
适用系统	awesom 视频处理IP

一、系统概览

VIP 仿真验证系统是一套围绕 awesom 视频处理 IP 的**端到端仿真工具链**，将图像/视频文件驱动为 Verilog 仿真激励，并将结果还原为可查看的图像/视频，同时提供量化分析。

一句话总结：给一张图进去，拿到一张图出来，中间经过你的 IP，最后告诉你 IP 处理得好不好。

核心能力

核心能力流程图

图1: VIP 仿真验证系统核心处理流程

二、环境准备

在开始使用本系统之前，请确保以下环境已就绪：

2.1 硬件要求

项目	最低配置	推荐配置
CPU	4 核	8 核以上

项目	最低配置	推荐配置
内存	8 GB	16 GB 以上
磁盘	5 GB 可用空间	20 GB 以上（用于存储测试视频）
操作系统	macOS / Linux / WSL2	64-bit 系统

2.2 Python 环境

版本要求：Python 3.8+

```
# 检查 Python 版本
python3 --version    # 应 ≥ 3.8

# （推荐）创建虚拟环境，避免依赖冲突
python3 -m venv venv
source venv/bin/activate
```

依赖包安装：

```
pip install pillow opencv-python numpy
```

包名	用途	说明
<code>pillow</code>	图像读写	PNG/BMP/JPG 等格式
<code>opencv-python</code>	颜色空间转换、视频拆帧/合成	图像分析核心依赖
<code>numpy</code>	数组计算	直方图、PSNR 等底层计算

注意：Apple Silicon (M1/M2/M3) 用户如果安装 `opencv-python` 失败，请使用 `pip install opencv-python-headless`。

验证 Python 环境：

```
python3 -c "import cv2; print('OpenCV version:', cv2.__version__)"
python3 -c "import numpy; print('NumPy version:', numpy.__version__)"
python3 -c "from PIL import Image; print('Pillow OK')"
```

2.3 Verilog 仿真器

工具链支持两种仿真器，**任选其一**即可。建议初学者使用 Icarus Verilog。

方式 A：Icarus Verilog（推荐，轻量级）

平台	安装命令
macOS	<code>brew install icarus-verilog</code>
Ubuntu/Debian	<code>sudo apt install iverilog</code>
Windows (WSL2)	<code>sudo apt install iverilog</code> （在 WSL 内执行）
源码编译	参见 https://github.com/steveicarus/iverilog

验证安装：

```
iverilog -V
# 预期输出: Icarus Verilog version 12.0 (stable) ...

vvp -V
# 预期输出: Icarus Verilog runtime engine ...
```

方式 B：Verilator（高性能，适合大规模仿真）

平台	安装命令
macOS	<code>brew install verilator</code>
Ubuntu/Debian	<code>sudo apt install verilator</code>
源码编译	参见 https://verilator.org/guide/latest/

验证安装：

```
verilator --version
# 预期输出: Verilator 5.x ...
```

选型建议：Icarus Verilog 安装简单、速度快，适合单帧图像仿真；Verilator 编译为 C++ 再运行，仿真速度更快，适合视频多帧连续仿真。

2.4 波形查看工具（可选，建议安装）

```
# macOS
brew install gtkwave

# Ubuntu/Debian
sudo apt install gtkwave
```

2.5 目录结构确认

克隆或下载本仓库后，确认以下目录存在：

```
cd video_sim
ls -la docs/ sim/ scripts/ output/
```

确保 `scripts/` 下所有 `.py` 文件有执行权限：

```
chmod +x scripts/*.py
```

2.6 环境自检脚本

运行以下脚本一键检查环境是否就绪：

```
echo "=== 环境自检 ==="

echo -n "Python: "; python3 --version
echo -n "OpenCV: "; python3 -c "import cv2; print(cv2.__version__)" 2>/dev/null || echo "未安装"
echo -n "NumPy: "; python3 -c "import numpy; print(numpy.__version__)" 2>/dev/null || echo "未安装"
echo -n "Pillow: "; python3 -c "import PIL; print(PIL.__version__)" 2>/dev/null || echo "未安装"
echo -n "iverilog: "; iverilog -V 2>/dev/null | head -1 || echo "未安装"
echo -n "verilator: "; verilator --version 2>/dev/null || echo "未安装（可选）"
echo -n "gtkwave: "; which gtkwave 2>/dev/null || echo "未安装（可选）"

echo "=== 自检完成 ==="
```

预期输出示例（无需全部匹配，Python 依赖 + 一个仿真器即可）：

```
=== 环境自检 ===
Python: Python 3.11.5
OpenCV: 4.9.0
NumPy: 1.26.4
Pillow: 10.2.0
iverilog: Icarus Verilog version 12.0 (stable)
verilator: 未安装 (可选)
gtkwave: /opt/homebrew/bin/gtkwave
=== 自检完成 ===
```

三、系统架构

系统架构图

图2: Python 工具链与 Verilog 仿真层双层架构

3.1 数据桥接

Python 和 Verilog 之间通过 **hex 文本文件** 桥接，不设运行时 IPC：

文件	方向	内容
stimulus.hex	Python → 仿真	32bit hex 像素数据
header.hex	Python → 仿真	分辨率、格式、帧数等元数据
result.hex	仿真 → Python	32bit hex 像素输出数据

3.2 支持的像素格式

格式 ID	名称	说明
3	RGB888	24bit 彩色，最常用
0	RAW8	8bit Bayer 原始
1	RAW10	10bit Bayer 原始
2	RAW12	12bit Bayer 原始

格式 ID	名称	说明
4	YUV422	16bit 色度子采样

四、完整案例：Gamma 校正 IP 仿真

本节以一个完整流程演示：输入一张图片，经过 Gamma 校正 IP，输出处理后的图片，然后分析 Gamma 曲线是否达标。

4.1 准备工作

```
# 进入项目根目录
cd video_sim

# 准备一张测试图像（以 1920×1080 的 RGB 彩色图为例）
cp ~/Pictures/test_1080p.png sim/testdata/input.png
```

4.2 第一步：生成仿真激励

```
python scripts/gen_stimulus.py \
    --input sim/testdata/input.png \
    --output sim/testdata/stimulus.hex \
    --format RGB888 \
    --width 1920 \
    --height 1080

python scripts/gen_header.py \
    --output sim/testdata/header.hex \
    --width 1920 \
    --height 1080 \
    --format RGB888 \
    --frames 1
```

`stimulus.hex` 内容示例（每行一个像素的 32bit hex 值）：

```
00FFA500 ← 像素 (0,0): R=FF, G=A5, B=00 → 橙色
0000FF00 ← 像素 (0,1): R=00, G=FF, B=00 → 绿色
...
```

4.3 第二步：运行仿真

```
make sim IP=gamma_corr WIDTH=1920 HEIGHT=1080 FORMAT=RGB888
```

仿真过程：

仿真过程序列图

图3: Gamma 校正 IP 仿真全流程时序图

4.4 第三步：还原输出图像

```
python scripts/gen_output.py \  
    --input sim/testdata/result.hex \  
    --output output/images/gamma_output.png \  
    --format RGB888 \  
    --width 1920 \  
    --height 1080
```

执行后打开 `output/images/gamma_output.png`，对比原图即可直观看到 Gamma 效果。

4.5 第四步：图像质量分析

```
python scripts/analyze.py \  
    --input sim/testdata/input.png \  
    --output output/images/gamma_output.png \  
    --all \  
    --report output/reports/gamma_report.json
```

分析报告示例：

```
{  
  "input_file": "input.png",  
  "output_file": "gamma_output.png",  
  "resolution": "1920x1080",  
  "format": "RGB888",  
  "timestamp": "2026-06-12 14:30:00",  
  "metrics": {  
    "psnr": {  
      "overall": 44.2,  
      "grade": "优秀"  
    },  
    "gamma": {
```

```

    "measured": 2.18,
    "target": 2.20,
    "delta": 0.02,
    "grade": "优秀"
  },
  "histogram": {
    "input_mean": 128.5,
    "output_mean": 186.3,
    "shift": "亮部提升, 符合 gamma=2.2 预期"
  }
},
"summary": "Gamma 校正效果优秀。实测 Gamma 值 2.18, 与目标 2.20 偏差仅 0.02。"
}

```

4.6 第五步（可选）：查看波形

```
make view_wave IP=gamma_corr
```

在 GTKWave 中检查 tvalid/tready 握手时序、tuser/tlast 信号和 tdata 数据流。

4.7 一键流程

```
make all IP=gamma_corr INPUT_IMG=sim/testdata/input.png
```

该命令自动执行 **激励生成** → **仿真** → **图像还原** → **质量分析** 全部四个步骤。

五、命令参考

5.1 Makefile 目标

目标	说明	示例
<code>make sim</code>	运行仿真	<code>make sim IP=gamma_corr</code>
<code>make gen_output</code>	结果 hex 还原为图像	<code>make gen_output</code> <code>IP=gamma_corr</code>
<code>make analyze</code>	图像质量分析	<code>make analyze IP=gamma_corr</code>

目标	说明	示例
<code>make all</code>	一键 sim → output → analyze	<code>make all IP=gamma_corr</code>
<code>make view_wave</code>	打开 GTKWave 查看波形	<code>make view_wave</code> <code>IP=gamma_corr</code>
<code>make test_all</code>	对所有 IP 运行全部测试	<code>make test_all</code>
<code>make clean</code>	清理输出	<code>make clean</code>

Makefile 变量：

变量	默认值	说明
<code>IP</code>	gamma_corr	目标IP名称
<code>SIM</code>	icarus	仿真器 (icarus/verilator)
<code>WIDTH</code>	1920	图像宽度
<code>HEIGHT</code>	1080	图像高度
<code>FORMAT</code>	RGB888	像素格式
<code>INPUT_IMG</code>	sim/testdata/input.png	输入图像路径

5.2 Python 脚本参数

gen_stimulus.py:

参数	说明	示例
<code>-i, --input</code>	输入图像路径	<code>input.png</code>
<code>-o, --output</code>	输出 hex 路径	<code>stimulus.hex</code>
<code>-f, --format</code>	像素格式	<code>RGB888</code>
<code>-w, --width</code>	目标宽度	<code>1920</code>

参数	说明	示例
<code>-h, --height</code>	目标高度	<code>1080</code>
<code>--fps</code>	帧率（多帧用）	<code>30</code>

analyze.py:

参数	说明	示例
<code>--input</code>	输入原图路径	<code>input.png</code>
<code>--output</code>	IP输出图路径	<code>output.png</code>
<code>--all</code>	运行全部分析项	
<code>--metric</code>	单项分析	<code>sharpness</code>
<code>--report</code>	报告输出路径	<code>report.json</code>
<code>--roi</code>	分析区域 (x,y,w,h)	<code>100,100,200,200</code>

5.3 支持的分析指标

指标	参数值	说明
锐度	<code>sharpness</code>	Laplacian/Sobel/Tenengrad/MTF
白平衡	<code>white_balance</code>	灰度世界增益/色温估算
PSNR	<code>psnr</code>	峰值信噪比，需提供参考图
Gamma 曲线	<code>gamma</code>	实测 vs 目标 Gamma 对比
直方图	<code>histogram</code>	RGB 三通道直方图分布
噪声	<code>noise</code>	高斯/椒盐噪声水平评估
色准	<code>color_checker</code>	24色卡 ΔE 分析（需色卡图像）

六、视频处理流程

处理视频时，系统逐帧拆分、逐帧仿真、再合成为视频。

```
# 1. 将视频拆帧并生成激励
python scripts/gen_video_stimulus.py \
    -i input.mp4 \
    -o sim/testdata/ \
    -f RGB888 -w 1920 -h 1080

# 2. 运行多帧仿真
make sim IP=denoise FRAMES=300

# 3. 合成输出视频
python scripts/gen_output.py \
    -i sim/testdata/result.hex \
    -o output/video/result.mp4 \
    -f RGB888 -w 1920 -h 1080 \
    --fps 30 --frames 300

# 4. 逐帧分析（取关键帧）
python scripts/analyze.py \
    --input input_keyframe.png \
    --output output_keyframe.png \
    --all
```

七、目录速查

video_sim/	
— docs/	
— DESIGN_SPEC.md	← 设计说明书
— USER_GUIDE.md	← 本文件
— sim/	
— common/	← 公共 BFM (4个模块)
— tests/	← 各 IP 的 Testbench
— testdata/	← 仿真数据 (hex 文件放这里)
— scripts/	← Python 工具集
— analyzers/	← 分析算法子模块
— utils/	← 像素格式、hex IO 工具
— output/	
— images/	← 输出图像
— reports/	← 分析报告 JSON
— waves/	← .vcd 波形文件

```
├─ Makefile
└─ README.md
```

八、常见问题

Q: 仿真报 "stimulus.hex not found"

A: 先运行 `python scripts/gen_stimulus.py ...` 生成激励文件，或使用 `make sim`（会自动调用 `gen_stimulus`）。

Q: 图像输出颜色不对

A: 检查 `--format` 参数是否与 IP 的实际数据格式一致。RGB888 和 YUV422 的字节排列不同。

Q: 分析工具报 "cv2 module not found"

A: `pip install opencv-python`，如果使用 ARM Mac 可能需要 `pip install opencv-python-headless`。

Q: 如何处理 4K 分辨率图像

A: `gen_stimulus.py` 会直接报分辨率超限，需确认目标 IP 支持 4K。修改 `--width 3840 --height 2160` 即可，但 hex 文件会很大（ $3840 \times 2160 = 8.3\text{M}$ 行）。

Q: 如何增加新的分析指标

A: 在 `scripts/analyzers/` 下新建模块，实现一个函数签名为 `def analyze(input_img: np.ndarray, output_img: np.ndarray = None) -> dict` 即可。`analyze.py` 会自动发现并注册。

文档版本：V1.0.0

最后更新：2026-06-12